The Compiler Verifies the Agents: Session-Typed Static and Runtime Verification for Multi-Agent LLM Coordination

Anonymous Author(s)
Affiliation
Address
email

Abstract

To verify an agent one must first fix a basis to verify it against—the oracle problem of software testing, which for LLM agents takes the form of a specification, a skill file, or an agent markdown. Existing instruments only check behavior against that basis. Runs exhibit single executions and LLM judges of contested reliability score traces; neither certifies the absence of failure. No system asks whether the basis itself is sound and complete as a coordination discipline, free of deadlock and able to reach its goal, so errors that are decidable statically surface only after runtime resources have been spent. We present STJP, a compiler-verifier for multi-agent coordination grounded in multiparty session types. An LLM drafts the user's intent as a Scribble global protocol; a static checker verifies it before any agent runs; and the validated type is projected into per-role contracts, deterministic monitors, guards, and a scheduler that enforce it on free-running, untrusted agents at the message boundary. Every metric is computed mechanically over a typed event log, without golden trajectories or LLM judges. The verifiers are themselves verified. Mutation testing detects 95.1% of seeded defects with zero false positives, the gate with value refinements blocks all 1,200 red-team exfiltration attempts, and we audit the fairness of our own harness. At n=100, unenforced protocol text reaches the goal in 100 of 100 trials while committing a forbidden irreversible act in 95 of them, whereas every enforcing arm shows zero disasters at every model tier and the full stack is also the cheapest, by 4–22$\times$ on cost to clean goal. A seeded circular-wait deadlock is rejected statically at zero token cost. Testing observes runs; typing quantifies over all of them.

1 Introduction

Verification presupposes a basis to verify against. In program verification that basis is the assertion of Hoare logic [Hoare, 1969]; in software testing it is the oracle, and the impossibility of judging observed behavior without one is the classical oracle problem [Barr et al., 2015]. For LLM agents the basis is today a specification, a policy, or a skill file, and the instruments that consult it are both downstream of execution. Benchmarks and trace evaluators measure, after the fact, how often agents violated policy [Levy et al., 2026, Wei et al., 2026], while LLM-as-judge pipelines score transcripts with a second model whose own reliability is contested—reported true-negative rates fall below 25% and expert agreement reaches only 64–68% [Masood et al., 2026]—and whose scores invite exactly the reward hacking this workshop names. Both observe executions against a verification basis, a spec or policy or skill file, whose own soundness nothing checks. No instrument asks whether the artifact the agents will faithfully follow can deadlock them, strand them short of the goal, or leave a

Submitted to the NeurIPS 2026 Workshop "Who Verifies the Agents? Toward Reliable Agent Development." Do not distribute.

role's duties hanging on a choice it cannot observe. Neither can certify a specification, and testing, as Dijkstra observed, shows the presence of bugs, never their absence [Dijkstra, 1972].

The gap matters most where agents meet each other. The largest empirical study of multi-agent failures, MAST, annotated 1,600+ traces across seven frameworks and found failures clustering not in model weakness but in *system design issues, inter-agent misalignment, and task verification* [Cemri et al., 2025]. The artifacts that fix this interaction discipline—`SKILL.md` files, `AGENTS.md` conventions, MCP manifests, enterprise agent specs [Anthropic et al., 2025]—are, functionally, programs. They say which role sends which message to which role, in what order, with which authorization preceding which irreversible act. Yet no tool verifies them. Format linting exists, and 34% of community skill packages violate even the published format spec [Saha et al., 2026], but format validity says nothing about interaction soundness. A skill can parse perfectly while describing a choreography in which the planner waits for a result the worker will never send, because the worker is waiting for an answer the planner cannot give. Nothing crashes; the system sits there, emitting tokens.

This paper answers the workshop's question in its static form. **The verifier is a compiler, and it runs before any agent does.** STJP (Session-Typed skills/Judge Protocol) is a pipeline built on multiparty session type theory [Honda et al., 2016, Scalas and Yoshida, 2019] and its protocol language Scribble [Honda et al., 2011]. The user states an intent; an LLM drafts a Scribble global protocol; the checker verifies well-formedness statically—rejecting deadlock precursors, incoherent choice, unguarded recursion, unreachable goals—and only a validated protocol is projected, role by role, into local contracts that become each agent's lean skill. From the same projection we mechanically generate a deterministic finite-state monitor per role, value-dependent choice guards in the asserted-MPST style [Bocchi et al., 2010], and a scheduler. The LLM-drafted protocol is an *agent-generated artifact under formal verification*; the agents stay free-running and untrusted, and the monitorability theorem [Bocchi et al., 2013] guarantees that local $O(1)$ boundary checks compose to global compliance—in contrast to concurrent work that obtains guarantees by generating the coordination layer as trusted code around opaque LLM calls [Bollig et al., 2026].

Three properties make this a verification-workshop contribution rather than only a systems one.

(1) The verification signal is deterministic and judge-free. Because every run is, by construction, a path through a validated global type, conformance and goal achievement receive mechanical, bit-reproducible definitions over a typed event log, with no hand-authored golden trajectory and no LLM judge (§4.1). The judged surface shrinks to what is genuinely semantic; a verifier that is plain code cannot be sweet-talked, and there is no scalar reward to game.

(2) The verifiers are themselves verified—including against us. We mutation-test the static checker against 626 seeded defects (95.1% well-formedness detection, zero false positives, one named residual gap), red-team the runtime gate with 1,200 injected exfiltration attempts (91.7%; with value refinements, 100%), hand-derive a 40-trace verdict corpus for monitor and grader, forensically audit all 1,200 benchmark trials, and—after asking *did we rig the grading?*—publish a fairness audit of our own harness that found five ways it had favored our system (§5).

(3) Verification stays faithful as agents evolve. A measured capability sweep across three Claude tiers shows that unenforced safety is capability-*dependent*, with premature irreversible filings going 95→0→0 on identical protocol text, while the enforcing arm remains capability-*invariant* at 100% clean-goal completion and zero disasters at every tier (§6). The closing gap is an average; the gate's zero is a guarantee.

These contributions map directly onto the workshop's pillars—robust, ungameable verifiers and red-teamed evaluation harnesses; runtime verification and benchmarks that stress-test the verification method itself; heterogeneous signals beyond scalar reward; formal verification of an agent-generated artifact; and Agent4Agent verification, since the drafter is an agent whose output governs other agents.

2 Related work

Governance layers and runtime enforcement. Recent syntheses organize agent governance as a three-layer stack [Liu et al., 2026, Ge et al., 2026]. Layer 0 checks that a specification parses and is commoditized [Saha et al., 2026]; Layer 2 measures whether runtime behavior matches claims and

Table 1: Positioning. STJP is, to our knowledge, the only system that both establishes guarantees *before* execution and enforces them on *untrusted* agents at runtime, with evaluation requiring neither a reference trajectory nor an LLM judge. "partial" = expressible per hand-written rule, without a global theorem. "n/a†" marks that in Bollig et al. [2026] agents cannot address messages at all, so recipient legality is moot rather than checked.

	When it acts	Deadlock-freedom	Goal reachability	Unauth. recip. blocked	Value/branch guards	Judge-free metrics
LLM-as-judge	post hoc	no	no	no	no	no
Rule guards [Wang et al., 2026]	runtime	no	no	partial	partial	no
Temporal/SMT [Agent-C, 2025]	runtime	no	no	partial	partial	no
Graph frameworks	wiring	no	no	no	no	no
MSC codegen [Bollig et al., 2026]	compile	static	no	n/a†	no	no
STJP (this work)	**compile + runtime**	**static**	**static**	**typed**	**yes**	**yes**

is an active benchmark frontier [Wei et al., 2026, Levy et al., 2026]; Layer 1, which asks whether the specification is semantically sound, is in those surveys' words "not available." STJP occupies it. Runtime guardrails act per execution. AgentSpec [Wang et al., 2026] enforces trigger/predicate/action rules around tool calls, Agent-C [Agent-C, 2025] compiles temporal constraints to SMT checks, and LGA [Ge et al., 2026] composes sandboxing and audit logging. Rule DSLs cannot express, let alone verify, *global* interaction properties—deadlock-freedom, goal reachability, choice coherence—because these are properties of the protocol structure, not of any single call. STJP rejects the faulty specification before the first LLM call, the only point at which a guarantee can be universal rather than per-run; its runtime gate then enforces exactly the properties the static phase proved enforceable [Bocchi et al., 2013]. We are explicit about where the novelty lies. The session-type theory is three decades deep and is assembled here, not invented; our contributions are the compiler-verifier framing, the gradual-typing integration of an untyped LLM participant [Igarashi et al., 2017], the observation that the scheduler is itself a projection artifact, and the judge-free empirical program this paper executes.

Projection-based coordination for LLM agents. Concurrently, Bollig et al. [2026] introduce an MSC-based DSL (ZipperGen) whose projection yields deadlock-free local programs by construction, machine-checked in Lean 4—a mechanization discipline we do not match and admire. The systems are separated by the trust boundary. In ZipperGen the LLM never holds the send keys, since coordination is trusted generated code—a clean closed-world result that cannot govern the deployed ecosystem, where free-running agents *are* the message-senders. STJP targets that open regime, in which the LLM is the untyped participant of gradual session types, the monitor is the cast at the boundary, and the operative theorem is monitorability; every phenomenon we measure exists only there, and Bollig et al. [2026] report no empirical evaluation. Certified MPST projection [Li et al., 2023, Li and Wies, 2025] is our named upgrade path; trace-based assurance [Paduraru et al., 2026] is downstream of the static question posed here.

Trace evaluation. Tool-and-step, process-utility [Chen et al., 2026], policy-conditioned [Levy et al., 2026], and reliability (pass^k) metrics all presuppose a hand-authored reference trajectory or an LLM judge, because an unconstrained agent leaves no structure to score a run against. Under STJP, the validated global type *is* the reference.

3 Verified properties, in sixty seconds

A *global type* G describes a whole conversation (roles, message labels, payload sorts, branching, guarded recursion); *projection* $\mathsf{G} \upharpoonright r$ extracts role r's local view, which induces a finite endpoint state machine (EFSM). A global type is *well-formed* if it projects onto every role and recursion is guarded; the merge condition rejects protocols in which a role's duties silently depend on a decision it was never told about—already a useful lint. Four classical theorems then transfer [Honda et al., 2016, Scalas and Yoshida, 2019, Bocchi et al., 2013]. **T1 communication safety** guarantees that no role ever receives a message it cannot handle; **T2 session fidelity** guarantees that every execution trace refines G; **T3 deadlock-freedom** guarantees that a well-typed single-session configuration never sticks; and **T4 monitorability** guarantees that if every endpoint is wrapped by a monitor enforcing its projected local type, the observable behavior of the whole *untrusted* network satisfies G, so local checks compose to a global guarantee with no central observer. T4 licenses the architecture—LLM

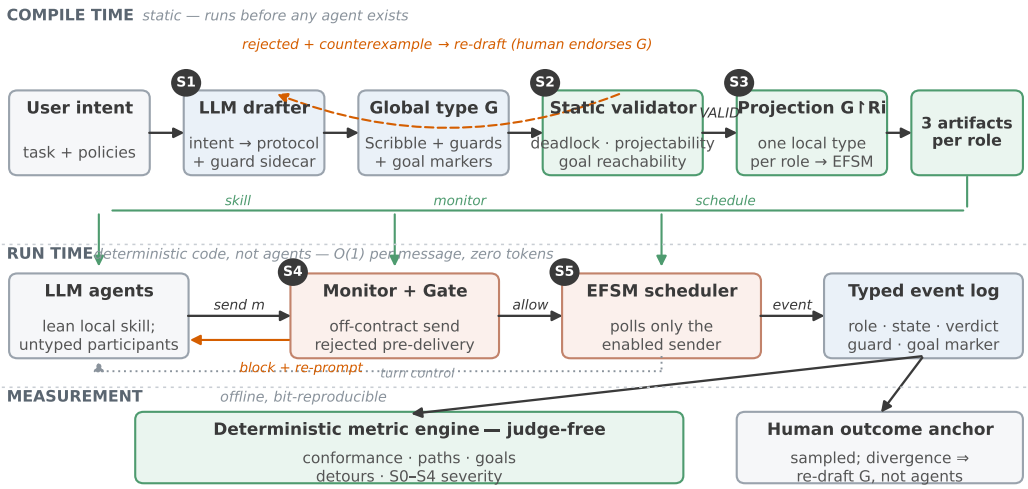

Figure 1: **The STJP pipeline.** At compile time an LLM drafts the user's intent as a Scribble global type with a guard sidecar; the static validator rejects unsafe drafts with counterexamples, and only a validated G is projected into three artifacts per role—a lean local contract, a generated EFSM monitor, and the schedule. At run time the gate rejects off-contract sends *pre-delivery* and re-prompts, the scheduler polls only the enabled sender, and every message lands in a typed event log over which all metrics are deterministic queries. One sampled human outcome anchor stays outside the metric family.

endpoints stay untrusted, monitors are generated from projections, and global compliance falls out of local, $O(1)$-per-event checking. Asserted/refined MPST [Bocchi et al., 2010, Chen and Honda, 2012] adds payload predicates and *choice guards*, closing the "protocol-legal but value-wrong" hole (skipping the audit on \$50k+ revenue is legal for classical MPST, a violation for guarded MPST). Appendix D tabulates the operational meaning of each guarantee.

4 The STJP verifier stack

Pipeline (Fig. 1). S1 An LLM drafts a Scribble global protocol from the user's intent, together with a sidecar of refinement and choice guards (`.refn`; stock `scribble-java` untouched) and *goal markers* whose ordered satisfaction encodes task completion. The draft is endorsed by the user, so the human sign-off point sits *before* any agent runs. **S2** The Scribble checker verifies well-formedness and the reachability of every goal marker, and the achievability witness doubles as the efficiency baseline. Verification is not decorative. While live-drafting the banking case, four consecutive drafts were rejected before the fifth passed; one contained a wait-for cycle that *would* deadlock at runtime, and agents run on that unsafe draft completed only 20% of trials. **S3** $\mathsf{G} \upharpoonright r$ is rendered as a lean per-agent skill, a SEND/RECV table with guards attached where decisions are made, often *smaller* than the prose it replaces (1,063–1,986 characters vs. 1,870 for the banking intent-only prompt). **S4** Each local type compiles to its EFSM, and the runtime monitor interprets it as a small generated deterministic program, $O(1)$ per event with no LLM call, issuing *allow*, *repair*, or *block* verdicts plus guard violations. In *gate* mode, off-contract sends are rejected pre-delivery and the role is re-prompted; by T4 these purely local gates jointly enforce G. **S5** The same automaton knows whose turn it is, so the scheduler prompts only roles enabled at the current state, targeting the dominant cost ($\sim$4.5 calls per delivered message unscheduled) and the dominant non-safety failure, liveness stalls.

4.1 Judge-free verification metrics

The monitor emits one typed event per message, recording role, protocol position, branch choice, gate verdict, guard outcomes, and goal-marker state. Over this log, evaluation is counting rather than judging. The *conformance rate* counts allowed, repaired, and blocked messages, computing Completion-under-Policy mechanically; the *path distribution* reduces each run to a named path

through G, where a retry-loop entry rate jumping from 9% to 40% is a regression no golden trajectory could encode; *goal progress* is read off the goal markers and is the only metric class that sees an agent looping in place; *detour cost* divides observed events by the compile-time witness's shortest path; and *repair burden* tracks gate interventions over time. All are bit-reproducible, since re-running the evaluators on a run directory yields identical numbers although the measured agents are stochastic.

Conformance has three levels. *Structural* (the trace is a path through G; deterministic), *refinement* (all payload and choice-guard predicates hold; deterministic), and *semantic* (payload content is faithful to the task)—only this residue may require an LLM judge, run batched, sampled, calibrated on human labels, and stored apart from the deterministic verdicts. The design principle is to **shrink the judged surface to what is genuinely semantic** and never let a judge score what a state machine can decide.

Goal achievement requires every goal marker in protocol order in the same attempt, under three critical-property checks covering data provenance (C1), context completeness (C2), and authorization before irreversible acts (C3). We additionally report **clean-goal completion (CGC)**, the fraction of trials that reach the goal with *zero* violations of any severity. GCR asks whether the system got there; CGC asks whether it got there without breaking anything on the way, and the gap between them is the lucky-run phenomenon that pass-rate metrics hide. One sampled human-labeled outcome measure stays *outside* the family as an anchor, because a perfectly conformant system can execute a badly conceived choreography; divergence between excellent protocol metrics and unhappy users indicts G and routes to re-drafting with the human, not to patching agents.

Consequence-graded violations (S0–S4). Raw "violation = deviation from our protocol" is circular—baselines that never saw the protocol fail by construction. We align observed labels semantically before judging, then grade each surviving deviation by *consequence*, from benign reorders (S0, not counted) and waste (S1) through skipped obligations (S2) and non-termination (S3) to the disaster class of an *irreversible act before its authorization* (S4). The taxonomy is itself validated. Across every arm of every run, $P(\text{goal failure} \mid \text{attempt had S2+}) = 100\%$, while benign deviations did not predict failure, so the grader measures harm rather than difference.

Arms. All arms share intent, role descriptions, model, and budgets, and the canonical validated type judges every arm. Arm **A** receives the intent only; **B** adds the validated global type pasted as text; **C** and **C-min** instead add the role's projected local contract (full or one-line-per-transition) under an observer monitor; and **C+** adds the *enforcement gate* and, where stated, the scheduler, forming the full STJP stack. Experiments were pre-registered, with predictions written before running and graded after (§7).

5 Verifying the verifiers

A verification paper that does not verify its own instruments has misunderstood its subject. Four exercises, in increasing order of self-exposure. Everything is measured; the full validation suite (1,200 ladder trials plus component experiments) cost under $100 of compute.

E0 — Test the testers first. A 40-trace verdict corpus with hand-derived expected verdicts (clean traces, every severity class, concurrent interleavings, unevaluable-guard silence, the `Approval(False)` edge case). Monitor and grader score 40/40—and, instructively, three of the *hand derivations* were wrong and corrected against the semantics; the instruments were right. All downstream experiments inherit this validated referee. An integration harness additionally exercises all five pipeline stages 100 times ($2105/2110 = 99.76\%$ checks passing; the five failures are a FIFO-swap mutation that yields another well-formed protocol, which Scribble *correctly* accepts).

E1 — Mutation-test the static checker (Fig. 2a). Seven mutation operators were applied over a 100-protocol corpus spanning 2–10 roles and four topologies, pre-registered to separate two defect families, and the data respected the split. *Well-formedness* defects are detected at 95.1% (226 mutants, with undeclared role and flipped-choice-subject at 100% and branch asymmetry at 82.5% raw) with 0/100 false positives on the valid corpus. Adjudicating the eleven branch-asymmetry survivors found seven *accidentally valid* (correct acceptances) and four sharing one genuine gap—a knowledge-of-choice deadlock the checker misses—re-scoping real-defect detection to 92.9% and converting a bare percentage into a *named limitation*. *Reordering* operators (400 mutants) are flagged at only 0.8%, which is correct behavior, because reordering an acyclic protocol usually produces a different

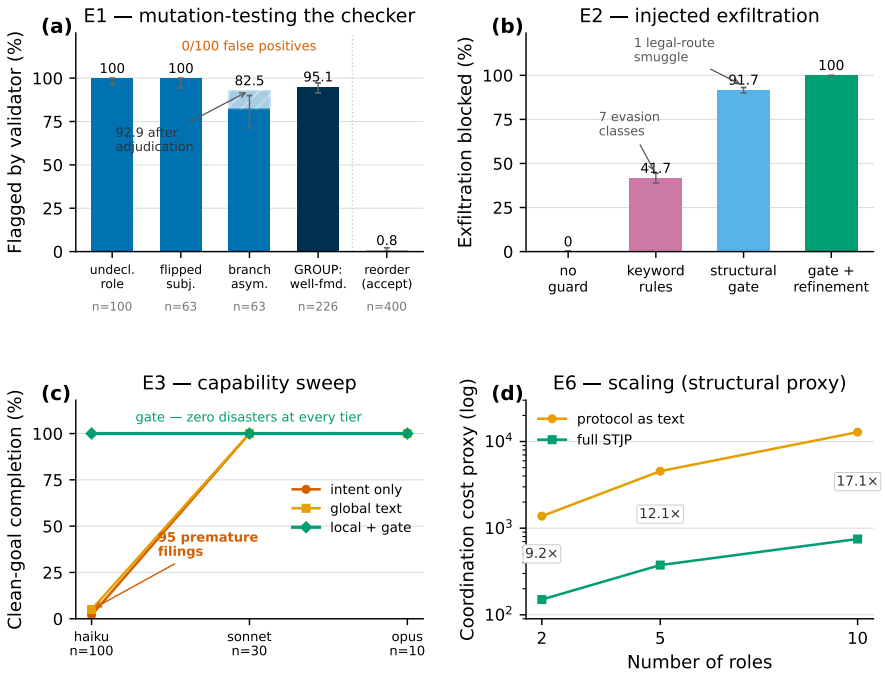

Figure 2: **Verifying the verifiers, measured.** (a) Mutation testing of the checker over a 100-protocol corpus. Well-formedness defects are detected at 95.1% with *zero* false positives on the valid corpus; flagging only 0.8% of reorderings is *correct acceptance*, since reordering an acyclic protocol usually yields another valid protocol. (b) Red-teaming with 1,200 injected exfiltration attempts per condition. Keyword rules reach 41.7% and are evaded by encodings, synonyms, and splitting; the structural gate reaches 91.7%, disguise-proof for every illegal route yet blind to a payload smuggled on a legal one; adding value refinements closes exactly that hole. The layering is the finding. (c) Capability sweep across three Claude tiers. Unenforced safety is capability-dependent, with 95 premature filings at the weak tier and zero above, while the gated arm stays flat at 100% CGC with zero disasters. (d) Structural cost proxy. The global-text-to-STJP coordination-cost ratio widens from $9.2\times$ at two roles to $17.1\times$ at ten.

but valid protocol; detecting *intent* drift under valid reorderings is a semantic scorer's job, not the type checker's, and conflating the two is how benchmarks overstate their tools. The standard here is deliberately the two-sided *preciseness* discipline of Chen et al. [2017, 2024]—soundness (nothing bad accepted) and completeness (nothing good rejected) measured together—transplanted from characterizing a subtyping relation to auditing a checker implementation.

E2 — Red-team the runtime gate (Fig. 2b). Twelve prompt-injection templates were run 100 times each against a finance protocol with a confidential payload and an ExternalAuditor role that legally receives only the public summary. With no guard, nothing is blocked. An honestly tuned keyword-and-role rule list blocks 41.7% and is evaded by encoding, synonyms, typos, split payloads, and innocuous relabeling. The structural gate blocks 91.7%, rejecting every attempt to use an illegal (label, recipient) edge *regardless of disguise* because legality is checked against the type rather than the wording; the surviving 8.3% is exactly one template that smuggles the payload *inside* a legitimately-labelled message on a legal route, which structure cannot see. The gate with value-refinement guards blocks 100%. Each mechanism catches precisely the class the theory says it catches, and none of them can be argued with, because none of them is a model.

Red-teaming our own harness (the fairness audit). A benchmark whose headline reads "0% vs. 100%" should expect the question of whether the grading was rigged, so we audited our own harness and report what it found, because the honest answer was "partly." Five findings led to five fixes. (1) The old headline rule asked bare agents to guess secret words. A trial passed only if the exact expected label crossed the exact expected edge, and the protocol arms were handed that vocabulary while the bare arm was not; grading it strictly is grading an essay by whether it contains the answer

key's exact sentence. The rule is now *per-arm* (protocol-shown arms judged strictly; bare arms pass on the right sender delivering a predicate-satisfying payload to the right receiver under any label), pre-audit tables are labeled, and bare-arm scores there are lower bounds. (2) Goal re-anchoring had silently eased two pass predicates; predicate changes are now blocked by an automated invariance guard unless a payload-type change forces a translation, which then requires human sign-off. (3) All arms shared one rate-limited deployment, so wall-clock figures flattered the arm making the fewest calls, which was ours; contended wall-clock is labeled indicative and speed claims are withheld, while the token and call counts that carry the cost claims are unaffected. (4) The scheduler's only opponent was round-robin poll-all, the weakest scheduler one can write. A protocol-free control that polls whoever just received a message is built and pre-registered with a deliberately falsifiable prediction, namely that it ties on linear pipelines and loses on branch and fan-in cases where "last receiver" has no answer. (5) The gate arm also whispered per-turn hints the observer arm never got, so a hint-free gate ablation is pre-registered, predicting that zero disasters are preserved while some liveness is lost. Evaluation harnesses are part of the trusted computing base of any verification claim; they deserve the same adversarial scrutiny as the system.

Forensic audit of the benchmark data. A post-hoc audit of all 1,200 ladder trials re-verified every suspicious pattern against raw state files and surfaced one material bug, namely 22 trials abandoned mid-play rather than failed, 18 of which depressed one gated arm's reported liveness to 79% against a true $\sim$97%. All 22 were driven to termination and every table regenerated; the 19 *genuine* gated-arm deadlocks were left standing, because replaying real failures until they pass is p-hacking. Scale also surfaced integrity incidents we log rather than hide, including subagent role-players caught writing auto-responder scripts instead of reasoning per poll (detected, discarded, and replayed), a round-numbering bug that could mask disasters, and a false-positive disaster check on enforcing arms, all fixed. The final state contains zero remaining fraud and zero non-terminal trials.

6 What the verifier finds at scale

Two tasks, `revenue_audit` (3 roles, the safety axis) and `escrow_trade` (4 roles, the cost axis), run six arms at n=100 each for 1,200 trials, turning three independent knobs—what the agent sees (intent, global text, or local projection), whether the monitor enforces (observe versus gate, the same program in two modes), and who gets polled (all roles every round versus only the enabled sender). Four findings follow (Fig. 3).

(i) Prose compliance does not survive concurrency. At smaller n, unenforced global text (B) had tied the gated arm on outcome and we declined to claim enforcement was necessary—a pre-registered prediction we graded as falsified (§7). At n=100 under concurrent polling the tie collapses. Arm B reaches the goal in 100% of `revenue_audit` trials *while filing before approval in 95 of 100*. Trace forensics make the mechanism exact—in 60 trials the Auditor approved without ever receiving the revenue figure, in 35 more all three messages fired in round one, and only 5 serialized. GCR conceals this entirely, while CGC reads 5%. This is the sharpest argument we know for verification signals richer than task success. **A scalar pass rate of 100% coexisted with a 95% disaster rate**, and only a structure-aware verifier could tell them apart.

(ii) Liveness must be driven, not observed. The observe-mode local-contract arm completes only 32% of `revenue_audit` trials—all 68 stalls show the same pattern, an agent re-sending into an unenforced wait. A contract can make every action that happens correct; it cannot, by observation alone, make an action happen. The projection must *drive* the runtime (the scheduler is a projection artifact), which is why runtime verification here is enforcement, not monitoring.

(iii) Safest and cheapest, simultaneously. Every enforcing arm shows zero disasters at n=100 in both tasks—not zero-so-far but zero *by construction*, since the gate rejects the disallowed send before delivery—and full STJP is also the cheapest arm (3.0 and 7.0 calls/trial vs. 3.3–28.8), because the scheduler never spends a call on a role whose turn it structurally is not. Table 2 prices the verification dividend. Dividing mean calls by CGC, B's apparently cheap 3.3 calls per trial becomes 66 calls per *clean* goal once poisoned successes stop counting, and full STJP wins by 4–22$\times$ on the metric an operator actually pays. In dollars, with weak-tier roles at list price, a clean and safe audit costs \$0.38 under STJP versus \$1.12–\$9.09 for the alternatives.

(iv) Verification remains faithful as agents evolve; unenforced safety does not. Which unenforced configuration fails is a function of model, task, and scheduling that nobody can predict in advance.

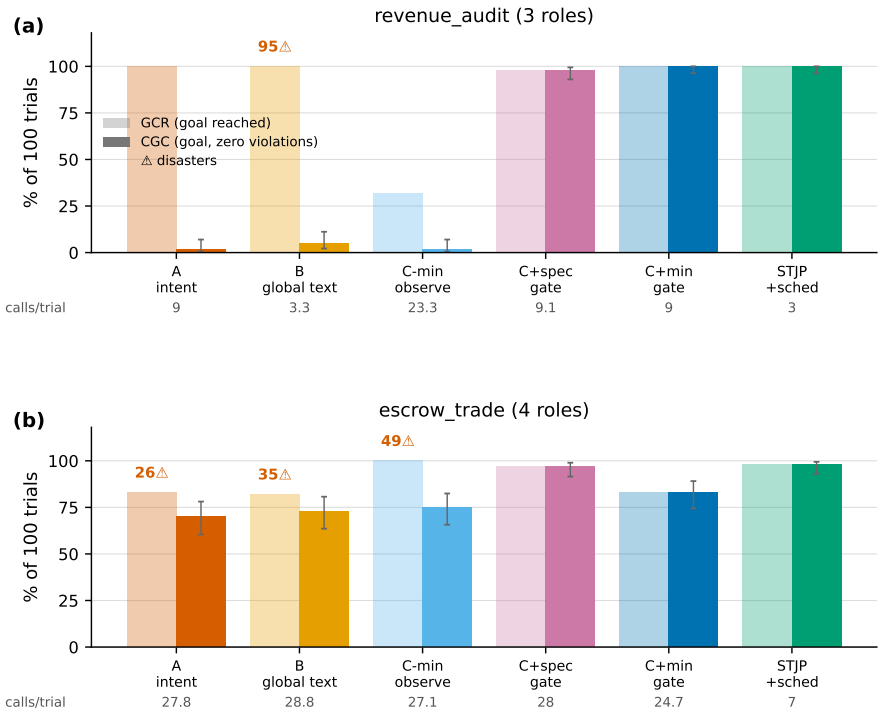

Figure 3: **The six-arm ladder at** n=100 (measured; 1,200 trials, cheap subagent roles, cost in LLM calls). Light bars show goal completion (GCR), solid bars clean-goal completion (CGC), red counts trials containing an unauthorized irreversible act, and the bottom row mean calls per trial. (a) On `revenue_audit`, where filing is allowed only after approval, unenforced global text reaches the goal in 100/100 trials yet files before approval in 95 of them, the observe-mode arm collapses to 32% on pure liveness, and every enforcing arm shows zero disasters *by construction*. (b) On `escrow_trade`, violations invisible at n=10 surface in every observe arm (26–49), and the scheduler delivers the cost win at 7.0 calls per trial against 24.7–28.8. Post-audit final numbers with no non-terminal trials.

Table 2: Cost to *clean* goal (mean calls/trial ÷ CGC), n=100 per arm. GCR-based cost rankings reverse once violated runs stop counting as successes.

Task	A intent	B global text	C-min obs.	C+spec	C+min	STJP
`revenue_audit`	450	66	1165	9.3	9.0	**3.0**
`escrow_trade`	39.7	39.5	36.1	28.9	29.8	**7.1**

Intent-only was the villain under a strong confident model, with 22 unauthorized irreversible acts in 30 attempts on the live finance case, so capability *amplified* the coordination gap; unenforced global text failed under concurrent polling (the 95 disasters above); and the observe-mode contract failed on liveness. The measured capability sweep (Fig. 2c, three Claude tiers, both tasks) closes the model axis. On identical protocol text, B's premature filings go 95→0→0 and intent-only's cleanliness goes 2%→100%, while the gated arm holds 100% CGC with zero disasters, and *zero gate rejections even needed*, at every tier. Read naively, the closing gap says strong models need no enforcement; read correctly, the gap is an *average* while the gate's zero is a *guarantee*, since production does not get to assume the strongest model and criticality is priced on the tail.

Deadlock and livelock, the failure classes runtime verification cannot rule out. A two-role protocol seeded with the classic circular wait stalls every bare run forever; Scribble rejects the global type *before any agent runs*, at zero token cost, and the repaired protocol executes at 100%. No runtime guardrail, judge, or benchmark can offer this, because they all require the failure to occur at least once. At n=100 the unchecked configuration deadlocks in 100/100 trials, burning 800 calls; STJP completes 100/100 at the protocol-optimal 7.0 calls/trial—*fewer total calls than the arm*

that never finishes. Nine live cases assembled from real, unmodified public skill files (Anthropic's skills repository, `awesome-copilot`, OpenAI Agents SDK, LangGraph, AutoGen, CrewAI, and AgenticPay; Table 3 in the appendix) replicate the shape in the wild. Seven of nine complete 0/10 as downloaded, and the remaining two complete only as a model-dependent coin flip whose direction *reverses* between Claude tiers. On the one looping protocol, the plan-as-text arm enters a *stable livelock*, with 10/10 trials burning the entire 16-round budget in motion and emitting 530 rule-breaking messages at $3.6\times$ the token cost of the STJP arm that merges 10/10 with zero violations. Deadlock is silence, livelock is motion without progress; the same static object rules out both, because both are properties of the protocol, not of any message.

7 Pre-registered predictions, graded, including the ones we got wrong

P1 (falsified twice, differently). We predicted that unenforced global text would slip below 100%. Under one model it did (40%); under a stronger one it did not, and the $n{=}100$ concurrent-polling run then showed the same arm at 100% GCR with 95 disasters, resolving what the tie had left open. What survives all outcomes is that the gate rejected real off-contract attempts in *every* gated run, and that enforcement value falls with model strength while the danger of no protocol at all rises. **P2 (confirmed beyond margin).** Scheduling saves at least 60% of tokens against lean+gate, with -65% tokens and -66% calls measured. **P3 (half-confirmed).** Projection alone does not beat global text, but projection with enforcement and scheduling dominates it. **P4 (deferred).** The orchestrator-cost comparison was inconclusive. **P5 (confirmed).** Scheduling cuts calls without inflating tokens per call. All eight validation-suite predictions were likewise registered and graded (E0–E2, E4–E7 CONFIRMED; E3's capability clause confirmed, vendor diversity open). We report the grades because a verifier that hides its own falsified predictions has misunderstood its subject.

8 Limitations

The verified artifact is not the verified intent. Intent→Scribble passes through an LLM; the validator rejects structurally invalid drafts, but a valid-yet-*unintended* protocol survives (validated $\neq$ intended). The shipped mitigations are human endorsement of G before any agent runs and the human outcome anchor. Because the grammar, validator, and an EFSM-equivalence scorer (300/300 on identical/reformatted/mutant pairs) exist, the translation step is a verifier-in-the-loop learning problem with a free deterministic reward; its training program is pre-registered and out of scope here. **Theorem scope.** Deadlock-freedom is per-session; interleaved sessions and dynamic role join/leave [Deniélou and Yoshida, 2011] are future work; our experiments fix the role set. **Expressiveness.** Open-ended brainstorming resists protocolization, and forcing it into a type would be dishonest; STJP targets the coordination-critical, authorization-laden regime. **Statistics and realism.** Live frontier-model runs remain $n{\leq}10$ per cell with wide Wilson intervals; the $n{=}100$ suite fixes power but substitutes cheap subagent role-players or contract-following strategies for frontier models. The capability sweep lacks a non-Claude frontier point (an access limitation we report rather than fake). Pre-audit finance tables use the strict label rule the fairness audit found unfair to bare arms (their scores are lower bounds; per-arm re-runs pending), and contended wall-clock figures carry no comparative claim. **No tools; invented data.** Roles carry no tools, so the C1 provenance check currently certifies message discipline, not real data lineage. **Grader and monitor.** The severity grader is payload-blind at milestones (the monitor-level guard closes this; the grader upgrade is pending); monitor correctness is validated, not mechanized—certified projection [Li and Wies, 2025] is our named upgrade path.

9 Conclusion

Who verifies the agents? For the coordination layer, a compiler can, before any agent runs and over all runs at once, with runtime enforcement and evaluation falling out of the same verified object as deterministic code. Unenforced arms fail in ways that move unpredictably with model, task, and scheduling, and their scalar pass rates actively conceal it, as a 100% goal rate coexisted with a 95% disaster rate; the enforcing arm is the only invariant of the grid, and the cheapest once violated successes stop counting. The verifiers are themselves mutation-tested, red-teamed, and audited—including the harness, including against us. Prompts move probabilities; only checks move guarantees.

References

Agent-C. Agent-C: Enforcing temporal constraints for LLM agents. *arXiv preprint arXiv:2512.23738*, 2025.

Anthropic, OpenAI, and MCP. Agent skills open standard; AGENTS.md convention; OpenAI model spec; model context protocol specification. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills; https://agents.md/; https://model-spec.openai.com/; https://modelcontextprotocol.io/specification/2025-11-25, 2025.

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

Laura Bocchi, Kohei Honda, Emilio Tuosto, and Nobuko Yoshida. A theory of design-by-contract for distributed multiparty interactions. In *Proc. International Conference on Concurrency Theory (CONCUR)*, 2010.

Laura Bocchi, Tzu-Chun Chen, Romain Demangeon, Kohei Honda, and Nobuko Yoshida. Monitoring networks through multiparty session types. In *Proc. FORTE; extended version in Theoretical Computer Science (2017)*, 2013.

Benedikt Bollig, Matthias Függer, and Thomas Nowak. Provable coordination for LLM agents via message sequence charts. *arXiv preprint arXiv:2604.17612*, 2026. ZipperGen; results machine-checked in Lean 4.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. arXiv:2503.13657.

Tzu-Chun Chen and Kohei Honda. Specifying stateful asynchronous properties for distributed programs. In *Proc. International Conference on Concurrency Theory (CONCUR)*, pages 209–224, 2012.

Tzu-Chun Chen, Mariangiola Dezani-Ciancaglini, Alceste Scalas, and Nobuko Yoshida. On the preciseness of subtyping in session types. *Logical Methods in Computer Science*, 13(2), 2017.

Tzu-Chun Chen, Mariangiola Dezani-Ciancaglini, and Nobuko Yoshida. On the preciseness of subtyping in session types: 10 years later. In *Proc. International Symposium on Principles and Practice of Declarative Programming (PPDP)*, pages 2:1–2:3, 2024.

Wei Chen et al. TRACE: Trajectory-aware comprehensive evaluation. In *Proc. The Web Conference (WWW)*, 2026.

Pierre-Malo Deniélou and Nobuko Yoshida. Dynamic multirole session types. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2011.

Edsger W. Dijkstra. The humble programmer. *Communications of the ACM*, 15(10):859–866, 1972.

Yun Ge et al. Governance architecture for autonomous agent systems (LGA). *arXiv preprint arXiv:2603.07191*, 2026.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Kohei Honda, Aybek Mukhamedov, Gary Brown, Tzu-Chun Chen, and Nobuko Yoshida. Scribble: Describing multiparty protocols. Scribble project; `scribble-java`, `nuScr`, `mpstk` toolchains, 2011.

Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):9:1–9:67, 2016. First presented at POPL 2008.

Atsushi Igarashi, Peter Thiemann, Yuya Tsuda, Vasco T. Vasconcelos, and Philip Wadler. Gradual session types. In *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP); extended version in JFP (2019)*, 2017.

Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. ST-WebAgentBench: Evaluating safety and trustworthiness in web agents. In *Proc. International Conference on Learning Representations (ICLR)*, 2026. arXiv:2410.06703.

Elaine Li and Thomas Wies. Certified implementability of global multiparty protocols. In *Proc. International Conference on Interactive Theorem Proving (ITP)*, 2025.

Elaine Li, Felix Stutz, Thomas Wies, and Damien Zufferey. Complete multiparty session type projection with automata. In *Proc. International Conference on Computer Aided Verification (CAV)*, 2023.

Tao Liu et al. A GovernSpec design framework for enterprise AI agents (contractual skills). *arXiv preprint arXiv:2605.22634*, 2026.

Masood, Deepchecks, and Autorubric. Surveys of LLM-as-judge reliability: Positional, verbosity, and self-preference biases; TNR $<25\%$; expert agreement 64–68%. Masood 2026; Deepchecks 2025; Autorubric, arXiv:2603.00077, 2026.

Ciprian Paduraru, Petru-Liviu Bouruc, and Alin Stefanescu. A trace-based assurance framework for agentic AI orchestration: Contracts, testing, and governance. *arXiv preprint arXiv:2603.18096*, 2026.

SafeRL-Lab (SAIL-Lab). AgenticPay: A buyer–seller negotiation benchmark for LLM agents. GitHub, MIT license. https://github.com/SafeRL-Lab/AgenticPay (commit 9740c5e), 2025.

Sourav Saha et al. Skilldex: A package manager for agent skills. *arXiv preprint arXiv:2604.16911*, 2026.

Alceste Scalas and Nobuko Yoshida. Less is more: Multiparty session types revisited. In *Proc. ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2019.

Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proc. International Conference on Software Engineering (ICSE)*, 2026. arXiv:2503.18666.

Xin Wei et al. BeSafe-Bench: Unveiling behavioral safety risks of situated agents in functional environments. *arXiv preprint arXiv:2603.25747*, 2026.

A A validated protocol and its guard sidecar

The abbreviated Scribble global type for the finance case follows.

```
global protocol Finance(role Ingestor, role RevenueAnalyst, role ExpenseAnalyst,
                        role TaxVerifier, role Auditor, role Writer) {
  RawRevenueData(Payload) from Ingestor to RevenueAnalyst;
  ExpenseData(Payload)    from ExpenseAnalyst to RevenueAnalyst;
  choice at RevenueAnalyst {
    HighRevenueNotification(Analysis) from RevenueAnalyst to Auditor;
    AuditReport(Findings)             from Auditor to TaxVerifier;
  } or {
    StandardRevenueNotification(Analysis) from RevenueAnalyst to TaxVerifier;
  }
  Approval(Decision)    from TaxVerifier to Writer;   // goal marker
  FinalReport(Document) from Writer to Ingestor;      // goal marker, FINAL
}
```

The choice-guard sidecar (`.refn`, asserted-MPST style, stock Scribble untouched) follows.

```
[choice at RevenueAnalyst]
when:    float(RawRevenueData) > 50000
require: HighRevenueNotification
over:    StandardRevenueNotification

[payload Approval]
assert:  Decision == 'true' => next in {FinalReport}   # denied-but-debited hole
```

The guard travels through four stages, from *define* (sidecar) to *state* (compiled into the contract at the decision state) to *check* (value-tracking monitor) to *enforce* (gate). Offline replay verification fires on high-revenue+standard-branch and on the symmetric case, and stays silent on correct branches and on unevaluable guards.

B Real-skills live runs

Table 3: Nine cases built from unmodified public agent/skill files, three arms each. "H"/"S" = Claude Haiku/Sonnet tiers in the two-model sweep (120 trials); *viol.* = rule-breaking messages. The `pr_review_merge` text arm is the measured livelock, completing 0/10 at $3.6\times$ the cost of the succeeding STJP arm. Every source file is deep-linked with license provenance in the repository. Two honesty notes apply. Earlier casts of two cases had misread their sources and were corrected, and runs are n=10 per arm with round-batched role-play, so same-role trials within a round share one model context.

Case	Source	No plan	Plan as text	Full STJP
`airline_seat`	OpenAI Agents SDK	0/10 deadlock	10/10, 10 dbl. writes	**10/10, 0, cheapest**
`booking_saga`	LangGraph	0/10 stall	10/10, 10 dbl. charges	**10/10, 0**
`code_execution`	AutoGen	0/10 deadlock	10/10	**10/10, cheapest**
`content_pipeline`	CrewAI (pattern)	0/10 stall	10/10	**10/10, cheapest**
`doc_pipeline`	anthropics/skills	flip: H 10/10, S 0/10	20/20, 120 viol.	**20/20 both, 0**
`pr_merge`	awesome-copilot	flip: H 0/10, S 10/10	20/20, 120 viol.	**20/20 both, 0**
`agenticpay_settlement`	AgenticPay	deadlock, all 3 tiers	—	**100%, all 3 tiers**
`pr_review_merge`	awesome-copilot	0/10 deadlock (r.2)	**0/10 livelock**, 530 viol.	**10/10, 0,** $3.6\times$ **cheaper**
`doc_coauthor_ship`	anthropics/skills	0/10, budget exhausted	10/10, 220 viol.	**10/10, 0,** $\sim$40% **cheaper**

The checker earned its keep on the way in. The corrected review protocol was itself debugged statically, with two Scribble rejections ("inconsistent external choice subjects" and "role progress violation") preceding validation—the §3 diagnostics firing on a realistic review loop before any agent ran. Adapting the buyer and seller agents of the MIT-licensed AgenticPay negotiation benchmark [SafeRL-Lab (SAIL-Lab), 2025] and adding an escrow settlement layer reproduces the classic circular-wait deadlock across three model tiers, while the validated escrow-first protocol completes.

Table 4: All five arms, one model, one run. "Harmful attempts" = attempts containing S2+/S4. GCR uses the pre-audit strict label rule for every arm (A's 0% is a lower bound; §5); Wilson 95% intervals at n=10 run from $[0, 28]\%$ at 0% to $[72, 100]\%$ at 100%. Seconds come from contended parallel execution and are indicative only.

Metric	A intent	B global	C local	C-min	C+ gate
GCR (strict)	0%	**100%**	60%	50%	**100%**
Attempts per trial	3.00	1.00	1.90	2.00	**1.00**
Delivered violations	180/180	26/100	16/151	15/153	**0/105**
S2 / S3 / **S4**	29 / 1 / **22**	0 / 0 / 0	0 / 13 / 0	0 / 15 / 0	**0 / 0 / 0**
Harmful attempts	**97%**	0%	0%	0%	**0%**
Tokens per trial	21.7k	24.4k	154.1k	84.0k	79.5k
Tokens per success	∞	**24.4k**	96.8k	44.7k	79.5k

C Live-model five-arm comparison (finance, gpt-5.4, n=10)

Under the stronger model, intent-only agents were *more* dangerous, not safer, committing 22 unauthorized irreversible acts in 30 attempts with 97% harmful attempts, against 4 disasters under the weaker model; capability amplifies the coordination gap. In the same run the gate intercepted five premature `Approval` sends; observed instead of gated, the identical mistake was delivered four times and once cascaded the session off-protocol. On the separate scheduler ablation (shared decentralized runtime, same task), protocol-as-text cost 120k tokens per delivered goal at 41.8 calls/trial, lean contract + gate 38k at 34.0, and full STJP **13.3k tokens per goal at 11.4 calls**—9× cheaper at identical 100% GCR and zero disasters, with tokens-per-call flat (1.12k vs. 1.07k), so the entire saving is fewer calls. The defeated opponent is round-robin poll-all; the pre-registered protocol-free scheduling control (§5) must be beaten on branching and fan-in cases before the scheduler claim is final.

Reliability and portability. With a contract-following strategy certifying the infrastructure at scale, the unchecked arm's Wilson interval narrows to $[0, 3.7]\%$ (structurally incapable, not unlucky) and STJP's pass^{10} at the Wilson floor rises from 0.039 to 0.686. The standalone monitor agrees with the in-process monitor on 59/59 testable canonical traces; a LangGraph adapter agrees on 14/14 clean-and-injected checks, consistent with T4's boundary placement.

D Guarantee transfer table

Table 5: Static guarantees inherited from MPST and their operational meaning for agent systems.

Guarantee	Agent-system meaning	Consequence
Deadlock-freedom (T3)	Circular waits rejected at compile time	Kills the silent-stall, token-burning failure mode for *all* runs
Communication safety (T1)	An agent can only emit messages the protocol permits at its state; recipient sets are part of the type	Off-protocol detours and unauthorized disclosure rejected by construction
Session fidelity (T2)	Every trace refines the endorsed G	The log is audit evidence against a proven reference
Monitorability (T4)	Local $O(1)$ gates compose to global compliance	Decentralized enforcement; no watching agents
Refinements + guards	Payload and branch predicates checked at the boundary	"Protocol-legal but value-wrong" branches become violations
Minimal capabilities	Required tools = labels of $\mathsf{G} \upharpoonright r$	Least-privilege `allowed-tools` derived, not guessed

E Reproducibility

All arms, cases, protocols, guards, run directories (events, prompts, per-message verdicts), the severity grader, the mutation and red-team suites, the fairness-audit review document, and the metric engine are in the accompanying repository (link withheld for double-blind review; provided to reviewers

via OpenReview supplementary material). Traces are the artifact of record, and re-running the deterministic evaluators on a run directory reproduces every table bit-for-bit. The total compute cost of the 1,200-trial validation suite was under $100.

F Responsible-use statement

STJP constrains agents rather than empowering them. Its outputs are rejections, monitors, and audit logs, so its dual-use surface is correspondingly small. The principal societal risk is *false assurance*, since "validated" is not "intended," and an operator who reads a checker pass as a guarantee of correct real-world behavior over-trusts the system, the same risk any verification tool carries. The mitigations shipped and evaluated here are human endorsement of the global type before any agent runs, a human-labeled outcome anchor that sits outside the metric family and routes divergence to protocol re-drafting, guarantees stated with their scope (per-session, fixed role set, structural versus semantic conformance), and prominent reporting of our own falsified predictions, checker gaps, and harness-fairness findings, so that the reliability claims can be audited rather than trusted. Enforcement infrastructure could in principle be used to make a *harmful* choreography reliable; the authorization-ordering and confidentiality guards evaluated here are, however, exactly the properties that make agent systems safer to deploy, and the human sign-off point is the natural place for organizational review.